DBMON-03: NoSQL Database Monitoring

> **Series:** DBMON — Database Monitoring | **Notebook:** 3 of 6 | **Created:** March 2026 | **Last Updated:** 03/12/2026

Overview

This notebook covers monitoring NoSQL databases with Dynatrace, including document stores (MongoDB, Cosmos DB), key-value stores (DynamoDB), and wide-column stores (Cassandra). You will learn how to analyze operation performance, read/write ratios, collection-level metrics, and replica health using distributed trace spans.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
OneAgent	Deployed on application hosts with NoSQL database clients
Permissions	`storage:spans:read`, `storage:entities:read`
Data	Application traffic generating NoSQL database calls
Prior Reading	DBMON-01: Database Monitoring Fundamentals

1. NoSQL Database Overview

NoSQL databases use different data models and operations than SQL databases. Dynatrace captures these differences through span attributes:

Database	Category	`db.system`	Common Operations	Key Characteristics
MongoDB	Document Store	`mongodb`	`find`, `insert`, `update`, `delete`, `aggregate`	BSON documents, replica sets
DynamoDB	Key-Value / Document	`dynamodb`	`GetItem`, `PutItem`, `Query`, `Scan`, `BatchWriteItem`	Partition keys, provisioned/on-demand capacity
Cassandra	Wide-Column	`cassandra`	`SELECT`, `INSERT`, `UPDATE`, `DELETE` (CQL)	Partition keys, consistent hashing
Cosmos DB	Multi-Model	`cosmosdb`	`ReadItem`, `CreateItem`, `Query`, `ReplaceItem`	Request Units (RU), multiple consistency levels
Couchbase	Document Store	`couchbase`	`get`, `upsert`, `query`, `search`	Memory-first, N1QL query language

Let's discover which NoSQL databases are active in your environment.

```
// Discover active NoSQL databases
fetch spans, from:-1h
| filter in(db.system, {"mongodb", "dynamodb", "cassandra", "cosmosdb",
"couchbase", "hbase"})
| summarize call_count = count(),
           avg_ms = avg(duration) / 1000000.0,
           p95_ms = percentile(duration, 95) / 1000000.0,
           unique_operations = countDistinct(db.operation),
           by:{db.system, server.address}
| sort call_count desc
```

2. MongoDB Monitoring

MongoDB is the most widely used document database. Dynatrace captures MongoDB operations through the official MongoDB driver instrumentation, providing visibility into query patterns, collection access, and replica set behavior.

Key MongoDB Span Attributes

Attribute	Description	Example
`db.system`	Always `mongodb`	`mongodb`
`db.operation`	MongoDB command	`find`, `insert`, `aggregate`
`db.namespace`	Database name	`orders`, `inventory`
`db.mongodb.collection`	Target collection	`users`, `products`
`server.address`	MongoDB host (or mongos for sharded)	`mongo-primary.internal`

```
// MongoDB operation breakdown by collection
fetch spans, from:-1h
| filter db.system == "mongodb"
| filter isNotNull(db.operation)
| summarize call_count = count(),
           avg_ms = avg(duration) / 1000000.0,
           p95_ms = percentile(duration, 95) / 1000000.0,
           by:{db.namespace, db.mongodb.collection, db.operation}
| sort call_count desc
| limit 25
```

```
// MongoDB slow operations — find operations exceeding 100ms
fetch spans, from:-1h
| filter db.system == "mongodb"
| filter duration &gt; 100000000
| fields timestamp, db.namespace, db.mongodb.collection, db.operation,
        db.statement, duration_ms = duration / 1000000.0
| sort duration_ms desc
| limit 20
```

```
// MongoDB operation volume over time
fetch spans, from:-6h
| filter db.system == "mongodb"
| filter isNotNull(db.operation)
| makeTimeseries ops = count(), by:{db.operation}, interval:5m
```

3. DynamoDB Monitoring

Amazon DynamoDB is a fully managed key-value and document database. Monitoring focuses on operation latency, consumed capacity, and identifying hot partitions through query patterns.

```
// DynamoDB operation performance by table
fetch spans, from:-1h
| filter db.system == "dynamodb"
| filter isNotNull(db.operation)
| summarize call_count = count(),
           avg_ms = avg(duration) / 1000000.0,
           p95_ms = percentile(duration, 95) / 1000000.0,
           errors = countIf(otel.status_code == "ERROR"),
           by:{db.namespace, db.operation}
| sort call_count desc
```

```
// DynamoDB Scan vs Query ratio — Scans are expensive and should be minimized
fetch spans, from:-1h
| filter db.system == "dynamodb"
| filter in(db.operation, {"Query", "Scan"})
| summarize op_count = count(),
           avg_ms = avg(duration) / 1000000.0,
           by:{db.operation}
| sort op_count desc
```

> **Warning:** DynamoDB `Scan` operations read every item in the table and consume significant read capacity. A high Scan-to-Query ratio indicates missing or underused indexes (Global Secondary Indexes). Aim for Query operations to dominate over Scans.

4. Cassandra Monitoring

Apache Cassandra uses CQL (Cassandra Query Language), which looks similar to SQL but has fundamentally different performance characteristics due to its distributed architecture.

```
// Cassandra operation breakdown by keyspace
fetch spans, from:-1h
| filter db.system == "cassandra"
| filter isNotNull(db.operation)
| summarize call_count = count(),
           avg_ms = avg(duration) / 1000000.0,
           p99_ms = percentile(duration, 99) / 1000000.0,
           by:{db.namespace, db.operation}
| sort call_count desc
```

```
// Cassandra latency spikes — detect operations exceeding 200ms
fetch spans, from:-6h
| filter db.system == "cassandra"
| makeTimeseries total = count(),
                 slow = countIf(duration &gt; 200000000),
                 interval:10m
```

5. Cosmos DB Monitoring

Azure Cosmos DB is a globally distributed multi-model database. Performance is measured in Request Units (RU), and monitoring focuses on operation latency, RU consumption patterns, and consistency-related behavior.

```
// Cosmos DB operation analysis
fetch spans, from:-1h
| filter db.system == "cosmosdb"
| filter isNotNull(db.operation)
| summarize call_count = count(),
           avg_ms = avg(duration) / 1000000.0,
           p95_ms = percentile(duration, 95) / 1000000.0,
           errors = countIf(otel.status_code == "ERROR"),
           by:{db.namespace, db.operation}
| sort call_count desc
```

6. Read vs Write Analysis

Understanding the read/write ratio is critical for NoSQL database tuning. Read-heavy workloads benefit from caching and read replicas, while write-heavy workloads may need partition optimization.

```
// Read vs Write ratio across all NoSQL databases
fetch spans, from:-1h
| filter in(db.system, {"mongodb", "dynamodb", "cassandra", "cosmosdb",
"couchbase"})
| filter isNotNull(db.operation)
| fieldsAdd rw_type = if(
    in(db.operation, {"find", "Query", "GetItem", "SELECT", "ReadItem",
"get", "search"}),
    then:"READ",
    else:"WRITE")
| summarize op_count = count(),
           avg_ms = avg(duration) / 1000000.0,
           by:{db.system, rw_type}
| sort db.system asc, rw_type asc
```

```
// Read/Write ratio trend over time — detect shifting workload patterns
fetch spans, from:-6h
| filter in(db.system, {"mongodb", "dynamodb", "cassandra", "cosmosdb"})
| filter isNotNull(db.operation)
| fieldsAdd rw_type = if(
    in(db.operation, {"find", "Query", "GetItem", "SELECT", "ReadItem",
"get"}),
    then:"READ",
    else:"WRITE")
| makeTimeseries op_count = count(), by:{rw_type}, interval:15m
```

7. Cross-Database Comparison

When your environment uses multiple NoSQL databases, comparing their performance characteristics helps identify which systems need attention.

```
// Compare NoSQL database performance — volume, latency, and error rates
fetch spans, from:-1h
| filter in(db.system, {"mongodb", "dynamodb", "cassandra", "cosmosdb",
"couchbase"})
| summarize total_calls = count(),
           avg_ms = avg(duration) / 1000000.0,
           p95_ms = percentile(duration, 95) / 1000000.0,
           error_count = countIf(otel.status_code == "ERROR"),
           unique_namespaces = countDistinct(db.namespace),
           by:{db.system}
| fieldsAdd error_rate_pct = round((toDouble(error_count) /
toDouble(total_calls)) * 100, decimals:2)
| sort total_calls desc
```

8. Summary and Next Steps

In this notebook you learned:

- How Dynatrace captures NoSQL database operations through span instrumentation
- MongoDB operation analysis by collection and performance profiling
- DynamoDB monitoring with Scan vs Query ratio analysis
- Cassandra latency analysis by keyspace and operation
- Cosmos DB operation monitoring
- Read vs Write ratio analysis for workload characterization
- Cross-database performance comparison techniques

Next Steps

- **DBMON-04: Cache and Messaging Monitoring** — Redis, Kafka, RabbitMQ, and Elasticsearch analysis
- **DBMON-05: Query Analysis** — Deep query analysis, N+1 detection, and optimization patterns

---